

Coding 1

Introduction to Python

ECBS5208 Central European University

Ulrich Wohak 16 September 2026

Welcome and course information

Your instructor

Ulrich Wohak

Postdoctoral Researcher in Economics
Central European University

Research interests

How managerial behaviour and firm dynamics shape macroeconomic outcomes.

Empirical industrial organization, applied microeconometrics,
and unstructured (text) data.

ulrichwohak.eu

wohaku@ceu.edu

Our teaching sessions

Six Wednesdays, 15:40–17:20

16 September to 21 October 2026

1. **A short quiz on the previous session**

At the start of Sessions 2–6.

2. **Explanation and live coding together**

Type your own code, predict results, and investigate errors.

3. **A ten-minute in-class assignment**

At the end of Sessions 2–6.

Today: setup, first code, and submission practice. No graded task.
Bring your laptop and a GitHub account. No programming prerequisite.

Assessment

20%

Five short quizzes

4% each.

Start of Sessions 2–6.

Closed-book, on the previous session.

20%

Five assignments

4% each.

End of Sessions 2–6.

Complete and submit during class.

60%

Final exam

Written and closed-book.

No computer.

**Friday, 6 November 2026
13:30**

Passing threshold: 50/100 overall.

All six scheduled meetings are teaching sessions.
The optional regression preview is outside the exam scope.

The in-class assignment

I provide a short Python script using the ideas from that day's class.

Your task is to explain the code in your own comments.

1. Follow the assignment link and **fork** the repository.
2. Work in your copy and add the explanations the task requests.
3. **Commit** your changes and **push** them to your fork.
4. Finish before the class ends. I collect the work from GitHub.

Today: share your GitHub username and practise this workflow.

At home: the notebooks in exercises/ are optional and ungraded.

Learning without AI in Coding 1

No AI tools for any coursework, including optional practice.

Support for learning

Course notebooks and your notes.

Official documentation and textbooks.

Questions in class and discussion of concepts.

Quizzes and the final exam are closed-book.

Coding 1 builds your own fluency. Coding 2 introduces AI-assisted work.

What this excludes

AI-generated code or explanations.

AI completion, translation, or rewriting.

Disable AI suggestions in your editor.

What you will be able to do

1. Read and write Python

Use variables and data structures, and explain how code behaves.

2. Work with data

Read files, clean tables with pandas, and check your transformations.

3. Make work reusable

Use conditions, loops, and functions. Diagnose common errors.

4. Explore and explain results

Produce summaries and graphs, and rerun a documented workflow.

The six-session roadmap

01	16 Sep	Setup, variables, expressions, and strings
02	23 Sep	Collections and reading and writing files
03	30 Sep	pandas tables and data cleaning
04	7 Oct	Plotting, conditions, and loops
05	14 Oct	Functions, exceptions, and descriptive statistics
06	21 Oct	Association, hypothesis tests, and course review

Each meeting builds on code and ideas we have already used.

Final exam: Friday, 6 November 2026 at 13:30, after the six teaching sessions.
Lecture folders are topic units. One meeting can use several folders.

Course scope

Our focus

- Python programming fundamentals
- Data handling and exploration
- A reproducible project environment
- The GitHub submission workflow

Beyond this course

- A full course in Git or the terminal
- SQL and other programming languages
- Advanced modelling and machine learning
- AI-assisted programming

We learn the setup and submission steps we need here.

Introductory regression is an optional, non-examinable preview of Coding 2.

What is Python?

A programming language for writing instructions that a computer can execute.

General purpose The same language can work with text, files, and data tables.

Libraries pandas supports table operations. matplotlib and plotnine create graphs.

Accessible tools Python is free and available on Windows, macOS, and Linux.

We use Python to practise programming through data problems.

A reproducible analysis workflow

Saved instructions connect the original data to the results.

Visible decisions	Code records the transformations. Comments explain the reasons.
Repeatable work	Rerun the same checks when the data change.
Work others can check	Share the inputs, code, and environment needed to recreate a result.

Manual edits and scattered spreadsheet formulas can make the steps hard to reconstruct. Clear code makes the sequence explicit.

A successful rerun still needs checks of the results.

The tools in our project

Python	The language we write.
JupyterLab	The interface for notebooks containing code, explanations, and output.
uv	Creates the project environment with the specified Python and packages.
Git and GitHub	Track changes and share repositories, including assignment submissions.

The place where you type matters.

Setup instructions go in the **terminal**.

Python instructions go in a notebook's **code cell**.

VS Code is an editor we can also use for scripts and a terminal.

Files, folders, and paths

Our course folder

Coding-1-Introduction-to-Python/	
lectures/	class notebooks
exercises/	optional practice
data/raw/	downloaded data
scripts/	setup helpers
pyproject.toml	project settings
uv.lock	package versions
.venv/	local environment

A **folder** contains files and subfolders.

The **project root** is the top folder shown here.

A **relative path** starts from the current working directory.

Different files hold different things:

.ipynb: notebook .py: Python script .csv: table data

Saving a file does not automatically create its parent folders.

Code, execution, and notebook state

A small Python example

```
price = 80  
nights = 3  
total = price * nights  
print(total)
```

Output: 240

Editing a cell does not execute it.

Before relying on a notebook, restart its kernel and run the cells from the top.

What happens in a notebook?

A **code cell** holds Python instructions.

Shift+Enter runs the selected cell.

The **kernel** executes the code and remembers values until it restarts.

Help and error messages

Reading help

`help(round)`

What does the function do?

What inputs does it accept?

What does it return?

Try a small example you can check.

Reading an error

Start with the final error message.

Find the line that caused it.

Check the names, types, and values.

Make one change, then run again.

When asking for help, show the code, the error, and what you expected.

Documentation is part of learning to program. Errors give us information.

Today's practical work

1. Open the course folder and check the Python environment.
2. Run a notebook cell, change an instruction, and inspect the result.
3. Practise variables, expressions, and strings together.
4. Make a first practice submission through your GitHub fork.

Today is ungraded. We make sure the workflow works for everyone.

Our first Python examples need no dataset.

Reference: course setup

After installing uv and cloning the course repository:

Open a terminal in the folder containing `pyproject.toml`.

```
uv sync
```

```
uv run python scripts/fetch_data.py
```

```
uv run jupyter lab
```

- These are **terminal commands**, not Python code cells.
- Open the first notebook and select the course's Python 3 kernel.
- Use `lectures/lecture00-intro/setup.md` for the full setup guide.

Reference: course materials and help

Course materials [Coding-1-Introduction-to-Python on GitHub](#)

Syllabus, session plan, lecture notebooks, and optional exercises

Python [Official Python 3.12 documentation](#)

In a notebook: `help(function_name)`

Data and graphs [pandas](#) [matplotlib](#) [plotnine](#)

Project tools [uv](#) [JupyterLab](#)

Course attribution Adapted teaching materials from *Coding for Data Analysis with Python*. See `NOTICE.md`.